```
            THEN
              Temp ← MyList[j]
              MyList[j] ← MyList[j + 1]
              MyList[j + 1] ← Temp
              NoMoreSwaps ← FALSE
          ENDIF
      NEXT j
      n ← n - 1
UNTIL NoMoreSwaps = TRUE
```

KEY TERM

Bubble sort: a sort method where adjacent pairs of values are compared and swapped

Discussion Point:

What happens if the array elements are already in the correct order?

TASK 13.05

Rewrite the algorithm in Worked Example 13.03 to sort the array elements into descending order.

13.05 Two-dimensional arrays

When we write a table of data (a matrix) on a piece of paper and want to refer to individual elements of the table, the convention is to give the row number first and then the column number. When declaring a 2D array, the number of rows is given first, then the number of columns. Again we have lower and upper bounds for each dimension.

In pseudocode, a 2D array declaration is written as:

```
DECLARE <identifier> : ARRAY[<lBound1>:<uBound1>,
<lBound2>:<uBound2>] OF <dataType>
```

The array elements in a 2D array can be numbered from 0. Sometimes it is more intuitive to use rows from row 1 and columns from column 1, as shown with the board game in Worked Example 13.04 and 13.05.

To declare a 2D array to represent a game board of six rows and seven columns, the pseudocode statement is:

```
Board : ARRAY[1:6,1:7] OF INTEGER
```

Accessing 2D arrays

A specific element in a table is accessed using an index pair. In pseudocode this is written as:

```
<arrayIdentifier>[x,y]
```

Pseudocode example:

```
Board[3,4] ← 0 // sets the element in row 3 and column 4 to zero
```

When we want to access each element of a 1D array, we use a loop to access each element in turn. When working with a 2D array, we need a loop to access each row. Within each row we need to access each column. This means we use a loop within a loop (nested loops).

In structured English our algorithm is:

```
For each row
   For each column
      Assign the initial value to the element at the current position
```

WORKED EXAMPLE 13.04

Working with two-dimensional arrays and nested loops

Using pseudocode, the algorithm to set each element of array `ThisTable` to zero is:

```
FOR Row ← 0 TO MaxRowIndex
    FOR Column ← 0 TO MaxColumnIndex
        ThisTable[Row, Column] ← 0
    NEXT Column
NEXT Row
```

We need the identifiers shown in Table 13.05.

Identifier	Data type	Explanation
`ThisTable`	`ARRAY[0:3, 0:5] OF INTEGER`	Table data structure (2D array) to store values
`MaxRowIndex`	`INTEGER`	The upper bound of the row index
`MaxColumnIndex`	`INTEGER`	The upper bound of the column index
`Row`	`INTEGER`	Counter for the row index
`Column`	`INTEGER`	Counter for the column index

Table 13.05 Identifier table for working with a table

When we want to output the contents of a 2D array, we again need nested loops. We want to output all the values in one row of the array on the same line. At the end of the row, we want to output a new line.

```
FOR Row ← 0 TO MaxRowIndex
    FOR Column ← 0 TO MaxColumnIndex
        OUTPUT ThisTable[Row, Column] // stay on same line
    NEXT Column
    OUTPUT Newline       // move to next line for next row
NEXT Row
```

TASK 13.06

1. Declare a 2D array to store the board data for the game Noughts and Crosses. The empty squares of the board are to be represented by a space. Player A's counters are to be represented by "O". Player B's counters are to be represented by "X".
2. Initialise the array to start with each square being empty.
3. Write a statement to represent player A placing their counter in the top left square.
4. Write a statement to represent player B placing their counter in the middle square.

WORKED EXAMPLE 13.05

Creating a program to play Connect 4

Connect 4 is a game played by two players. In the commercial version shown in Figure 13.06, one player uses red tokens and the other uses black. Each player has 21 tokens. The game board is a vertical grid of six rows and seven columns.

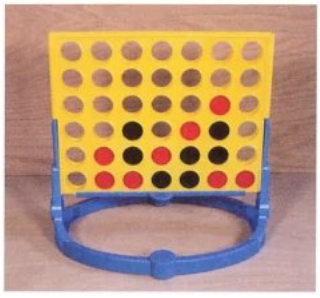


Figure 13.06 A Connect 4 board

Columns get filled with tokens from the bottom. The players take it in turns to choose a column that is not full and drop a token into this column. The token will occupy the lowest empty position in the chosen column. The winner is the player who is the first to connect four of their own tokens in a horizontal, vertical or diagonal line. If all tokens have been used and neither player has connected four tokens, the game ends in a draw.

If we want to write a program to play this game on a computer, we need to work out the steps required to 'solve the problem', that means to let players take their turn in placing tokens and checking for a winner. We will designate our players (and their tokens) by 'O' and 'X'. The game board will be represented by a 2D array. To simplify the problem, the winner is the player who is the first to connect four of their tokens horizontally or vertically.

Our first attempt in structured English is:

```
Initialise board
Set up game
Display board
While game not finished
   Player makes a move
   Display board
   Check if game finished
   If game not finished, swap player
```

The top-level pseudocode version using modules is:

```
01 CALL InitialiseBoard
02 CALL SetUpGame
03 CALL OutputBoard
04 WHILE GameFinished = FALSE DO
05     CALL PlayerMakesMove
06     CALL OutputBoard
07     CALL CheckGameFinished
08     IF GameFinished = FALSE
```

(Continued)

```
09        THEN
10          CALL SwapThisPlayer
11      ENDIF
12  ENDWHILE
```

Note that Steps 03 and 06 are the same. This means that we can save ourselves some effort. We only need to define this module once, but can call it from more than one place. This is one of the advantages of using modules.

The identifier table for the program is shown in Table 13.06.

Identifier	Data type	Explanation
`Board`	`ARRAY[1:6,1:7] OF CHAR`	2D array to represent the board
`InitialiseBoard`		Procedure to initialise the board to all blanks
`SetUpGame`		Procedure to set initial values for `GameFinished` and `ThisPlayer`
`GameFinished`	`BOOLEAN`	FALSE if the game is not finished TRUE if the board is full or a player has won
`ThisPlayer`	`CHAR`	`'O'` when it is Player O's turn `'X'` when it is Player X's turn
`OutputBoard`		Procedure to output the current contents of the board
`PlayerMakesMove`		Procedure to place the current player's token into the chosen board location
`CheckGameFinished`		Procedure to check if the token just placed makes the current player a winner or board is full
`SwapThisPlayer`		Procedure to change player's turn

Table 13.06 Initial identifier table for Connect 4 game

Now we can refine each procedure (module). This is likely to add some more identifiers to our identifier table. The additional entries required are shown after each procedure.

```
PROCEDURE InitialiseBoard
    FOR Row ← 1 TO 6
        FOR Column ← 1 TO 7
            Board[Row, Column] ← BLANK // use a suitable value for blank
        NEXT Column
    NEXT Row
ENDPROCEDURE
```

Identifier	Data type	Explanation
`Row`	`INTEGER`	Loop counter for the rows
`Column`	`INTEGER`	Loop counter for the columns
`BLANK`	`CHAR`	A value that represents an empty board location

Table 13.07 Additional identifiers for the `InitialiseBoard` procedure

(Continued)

```
PROCEDURE SetUpGame
    ThisPlayer ← 'O' // Player O always starts
    GameFinished ← FALSE
ENDPROCEDURE

PROCEDURE OutputBoard
    FOR Row ← 6 DOWNTO 1
        FOR Column ← 1 TO 7
            OUTPUT Board[Row, Column] // don't move to next line
        NEXT Column
        OUTPUT Newline // move to next line
    NEXT Row
ENDPROCEDURE

PROCEDURE PlayerMakesMove
    ValidColumn ← PlayerChoosesColumn // a module returns column number
    ValidRow ← FindFreeRow // a module returns row number
    Board[ValidRow, ValidColumn] ← ThisPlayer
ENDPROCEDURE
```

Identifier	Data type	Explanation
ValidColumn	INTEGER	The column number the player has chosen
PlayerChoosesColumn	INTEGER	Function to get the current player's valid choice of column
ValidRow	INTEGER	The row number that represents the first free location in the chosen column
FindFreeRow	INTEGER	Function to find the next free location in the chosen column

Table 13.08 Additional identifiers for the `PlayerMakesMove` procedure

```
FUNCTION PlayerChoosesColumn RETURNS INTEGER// returns a valid column number
    OUTPUT "Player ", ThisPlayer, "'s turn."
    REPEAT
        OUTPUT "Enter a valid column number: "
        INPUT ColumnNumber
    UNTIL ColumnNumberValid = TRUE // check whether the column number is valid
    RETURN ColumnNumber
ENDFUNCTION
```

Identifier	Data type	Explanation
ColumnNumber	INTEGER	The column number chosen by the current player
ColumnNumberValid	BOOLEAN	Function to check whether the chosen column is valid

Table 13.09 Additional identifiers for `ThisPlayerChoosesColumn` function

Note that we need to define the function `ColumnNumberValid`. A column is valid if it is within the range 1 to 7 inclusive and there is still at least one empty location in that column.

(Continued)

```
FUNCTION ColumnNumberValid RETURNS BOOLEAN
  // returns whether or not the column number is valid
    Valid ← FALSE
    IF ColumnNumber >= 1 AND ColumnNumber <= 7
      THEN
        IF Board[6, ColumnNumber] = BLANK // at least 1 empty space in column
          THEN
            Valid ← TRUE
        ENDIF
    ENDIF
    RETURN Valid
ENDFUNCTION
```

Identifier	Data type	Explanation
Valid	BOOLEAN	FALSE if column number is not valid TRUE if column number is valid

Table 13.10 Additional identifier for the ColumnNumberValid function

```
FUNCTION FindFreeRow RETURNS INTEGER
  // returns the next free position
    ThisRow ← 1
    WHILE Board[ThisRow, ValidColumn] <> BLANK DO // find first empty cell
        ThisRow ← ThisRow + 1
    ENDWHILE
    RETURN ThisRow
ENDFUNCTION
```

Identifier	Data type	Explanation
ThisRow	INTEGER	Points to the next row to be checked

Table 13.11 Additional identifier for the FindFreeRow function

```
PROCEDURE CheckGameFinished
    WinnerFound ← FALSE
    CALL CheckIfPlayerHasWon
    IF WinnerFound = TRUE
      THEN
        GameFinished ← TRUE
        OUTPUT ThisPlayer " is the winner"
      ELSE
        CALL CheckForFullBoard
    ENDIF
ENDPROCEDURE
```

Note that the CheckGameFinished procedure uses two further procedures that we need to define.

(Continued)

Identifier	Data type	Explanation
WinnerFound	BOOLEAN	FALSE if no winning line TRUE if a winning line is found
CheckIfPlayerHasWon		Procedure to check if there is a winning line
CheckVerticalLineInValidColumn		Procedure to check if there is a winning vertical line in the column the last token was placed in
CheckForFullBoard		Procedure to check whether the board is full

Table 13.12 Additional identifiers for the CheckGameFinished procedure

```
PROCEDURE CheckIfPlayerHasWon
    WinnerFound ← False
    CALL CheckHorizontalLine
    IF WinnerFound = FALSE
      THEN
        CALL CheckVerticalLine
ENDPROCEDURE
PROCEDURE CheckHorizontalLine
    FOR i ← 1 TO 4
       IF Board[ValidRow, i] = ThisPlayer AND
         Board[ValidRow, i + 1] = ThisPlayer AND
         Board[ValidRow, i + 2] = ThisPlayer AND
         Board[ValidRow, i + 3] = ThisPlayer
         THEN
           WinnerFound ← TRUE
       ENDIF
    NEXT i
ENDPROCEDURE

PROCEDURE CheckVerticalLine
    IF ValidRow = 4 OR ValidRow = 5 OR ValidRow = 6
      THEN
        IF Board[ValidRow, ValidColumn] = ThisPlayer AND
          Board[ValidRow - 1, ValidColumn] = ThisPlayer AND
          Board[ValidRow - 2, ValidColumn] = ThisPlayer AND
          Board[ValidRow - 3, ValidColumn] = ThisPlayer
          THEN
            WinnerFound ← TRUE
        ENDIF
    ENDIF
ENDPROCEDURE

PROCEDURE CheckForFullBoard
    BlankFound ← FALSE
    ThisRow ← 0
    REPEAT
        ThisColumn ← 0
        ThisRow ← ThisRow + 1
        REPEAT
```

(Continued)

```
            ThisColumn ← ThisColumn + 1
            IF Board[ThisRow, ThisColumn] = BLANK
              THEN
                BlankFound ← TRUE
            ENDIF
        UNTIL ThisColumn = 7 OR BlankFound = TRUE
    UNTIL ThisRow = 6 OR BlankFound = TRUE
    IF BlankFound = FALSE
      THEN
        OUTPUT "It is a draw"
        GameFinished ← TRUE
    ENDIF
ENDPROCEDURE
```

Identifier	Data type	Explanation
BlankFound	BOOLEAN	FALSE if no blank location found on the board TRUE if a blank location found on the board
ThisRow	INTEGER	Loop counter for rows
ThisColumn	INTEGER	Loop counter for columns

Table 13.13 Additional identifiers for the `CheckForFullBoard` procedure

```
PROCEDURE SwapThisPlayer
    IF ThisPlayer = 'O'
      THEN
        ThisPlayer ← 'X'
      ELSE
        ThisPlayer ← 'O'
    ENDIF
ENDPROCEDURE
```


We can also use arrays of records. Using the Person record type from Section 13.02, we can declare an array of that type for 100 person records:

```
DECLARE Person : ARRAY[1:100] OF PersonType
```

We can then access an individual's data. For example the first person's name in the array is set as follows:

```
Person[1].Name ← "Fred"
OUTPUT Person[1].Name
```

This is particularly useful when we have several people's data to work with and do not want to use a separate 1D array for each field.

TASK 13.07

1 Declare an array of BookType (see Task 13.02) for 200 books.
2 Set the first book's details to the values given in Task 13.02.